

INSTITUTO FEDERAL DO ESPÍRITO SANTO - IFES

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Alice Lourenço, Davi Sutil e Karyna Carbas

Trabalho de Técnicas de Programação Avançada

**Trabalho de Implementação de Listas Genéricas e Análise
de Complexidade de Listas**

SERRA - ES

2026

Atribuições e Responsabilidades					
Atividade	Implementação de uma biblioteca de Listas em Java usando Generics	Prática de Implementação: Implementação de um programa que use a biblioteca	Análise Matemática de complexidade dos Algoritmos	Análise Empírica de complexidade de Algoritmos	Produção, Revisão e Estruturação de Relatório
Aluno	Davi Sutil	Davi Sutil	Alice Lourenço e Karyna Carbas	Alice Lourenço, Davi Sutil e Karyna Carbas	Alice Lourenço, Davi Sutil e Karyna Carbas
Tempo Gasto (Horas)	7	4	3	26	3
Observações				Aproximadamente 23h do tempo total foram voltadas para os testes, mas eles não foram realizados todos de forma sequencial. Essas horas foram diluídas.	

Link do repositório: <https://github.com/DaviCSS/trabalhoTPA>

1. Introdução

O presente trabalho desenvolvido pelos alunos Alice Lourenço, Davi Sutil e Karyna Carbas, é parte das atividades que compõem a disciplina de Técnicas de Programação Avançadas do curso de Bacharelado em Sistemas da Informação do Instituto Federal do Espírito Santo - Campus Serra.

O objetivo central deste trabalho é revisar e aplicar conceitos fundamentais de estruturas de dados e programação orientada a objetos em Java, com foco na implementação de listas encadeadas genéricas e na análise de complexidade de algoritmos. Para isso, o trabalho está dividido em três etapas principais que seguem as orientações do professor Victório Albani, sendo estas etapas, (i) o desenvolvimento de um programa de teste que utiliza essa biblioteca para manipular dados de contato, (ii) análise matemática da complexidade dos principais métodos da biblioteca e (iii) a realização de testes empíricos para avaliar o desempenho das operações implementadas em função do tamanho da entrada. Nos testes empíricos será

considerado o pior caso de execução das operações de busca e remoção, utilizando como referência o último elemento da lista, dessa forma, serão realizadas buscas pelo nome do último contato e registrando-se o tempo de execução de cada operação para diferentes tamanhos de entrada.

Além disso, o relatório que documenta detalhadamente todas as etapas do desenvolvimento, assim sendo, a seção de inteligência artificial (IA), tem como objetivo garantir transparência quanto às ferramentas utilizadas e à forma como contribuíram para o resultado final.

Dessa forma, o trabalho busca não apenas consolidar conhecimentos teóricos e práticos sobre listas encadeadas e análise de complexidade, mas também desenvolvimento de competências relacionadas à documentação técnica, construção de raciocínio lógico, trabalho em equipe e o uso crítico de tecnologias emergentes no processo de aprendizagem.

2. O uso de Inteligência Artificial

Durante o desenvolvimento deste trabalho, a inteligência artificial foi empregada como uma ferramenta de apoio em diversas etapas, abrangendo desde a implementação técnica até a documentação final. No escopo da programação, a IA auxiliou na construção da lógica para a sobrescrição do método equals, permitindo que a mesma função realizasse buscas tanto por nome quanto por telefone, além de contribuir na elaboração da função de leitura dos arquivos de entrada e no desenvolvimento do menu interativo na classe principal. Para viabilizar os testes empíricos, a ferramenta foi essencial na geração dos documentos de entrada, criando massas de dados aleatórios em diferentes escalas. Já no aspecto visual e de estruturação, o uso da tecnologia permitiu a criação das imagens dos grafos, a melhoria na apresentação visual dos trechos de código e das fórmulas utilizadas, a estruturação básica do arquivo README.md e, por fim, o aprimoramento geral da fluidez textual do documento.

3. Análise Matemática de complexidade dos Algoritmos

A análise matemática de algoritmos complexos, busca descobrir como a quantidade de operações cresce conforme aumenta-se o tamanho das entradas. Para isso, representamos o tamanho das entradas por n e analisamos quantas vezes cada operação é executada. A partir disso, podemos construir uma função $T(n)$ e identificar sua ordem de crescimento do algoritmo.

Para expressar esses limites, normalmente utiliza-se a notação de **Big O** ou "**Ó de Landau**", que refere-se à notação de Landau (ou notação assintótica), um conjunto de símbolos matemáticos criados por Paul Bachmann e Edmund Landau. Ela serve para descrever o comportamento e os limites de uma função quando seu argumento tende a um determinado valor ou ao infinito $O(n)$.

Essa análise é importante porque permite a comparação de algoritmos e prever quais deles terão melhor desempenho quando trabalharmos com grandes volumes de dados. Durante o trabalho, realizaremos as análises de carregamento, busca, remoção e contagem de nós, sempre analisando o pior caso.

a. Análise de Lista Não Ordenada

Uma Lista Encadeada Não Ordenada é uma estrutura de dados dinâmica composta por nós encadeados em sequência unidirecional. A **inserção** de um novo elemento possui comportamentos assintóticos distintos a depender da estratégia adotada pela classe *ListaEncadeada<T>*, como demonstrado no trecho a seguir:

```
Java
1 private void adicionarNaoOrd(T elem) {
2     No<T> novo = new No<T>(elem); //c1: alocação + c2: atribuição
3     if (this.prim == null) { //c3: avaliação condicional
4         this.prim = this.ult = novo; // c4: atribuição dupla (se lista vazia)
5     } else {
6         this.ult.setProx(novo); // c5: atribuição de referência (se lista com nós)
7         this.ult = novo; // c6: redefinição do ponteiro cauda
8     }
9     this.quant++; // c7: incremento (leitura + soma + escrita)
10 }
```

No método Não Ordenado *adicionarNaoOrd*, a linha 2 instancia um novo objeto do tipo *No<T>*, exigindo um custo constante de alocação de memória ($c1 + c2$). A linha 3 realiza a avaliação condicional da instrução *this.prim == null* ($c3$).

Caso a estrutura esteja vazia, a linha 4 atribui a referência do novo nó simultaneamente às variáveis *prim* e *ult* ($c4$) resultando no tempo $T(0) = c1 + c2 + c3 + c4 + c7$. Caso a lista já contenha elementos, a execução é desviada para o bloco *else*, onde a linha 6 invoca *this.ult.setProx(novo)* para encadear o novo nó no final da estrutura ($c5$) e a linha 7 atualiza a referência de cauda executando *this.ult = novo* ($c6$), resultando no tempo $T(n) = c1 + c2 + c3 + c5 + c6 + c7$. Finalmente, a linha 9 incrementa a variável de controle *quant* ($c7$).

Como o método utiliza a referência explícita do último elemento (*ult*), não há necessidade de utilizar laços de repetição como *while* ou *for*. Todas as instruções são executadas em uma sequência fixa de passos.

Para que a explicação fique mais visível, vamos pensar em dois cenários:

1. A lista tem 0 elementos: O código executa as linhas 2, 3, 4 e 9. Isso gasta, por exemplo, 10 nanossegundos.
2. A lista tem 800.000 de elementos: O código cai no *else* e executa as linhas 2, 3, 6, 7 e 9. Isso gasta, por exemplo, 12 nanossegundos.

Se definirmos nosso teto máximo $C = 12$ nanossegundos, podemos afirmar:

Tempo do código $T(n) \leq 12 * 1$ nanossegundos

Dessa forma, definindo $C = \max(T(0), T(n))$, o tempo total é limitado por $T(n) \leq C * 1$. O pior caso ocorre sob qualquer condição de preenchimento da lista, resultando em uma complexidade de tempo constante, representada por $O(1)$.

Entendendo como funciona a inserção de dados na lista não ordenada, passamos para o método responsável por pesquisar.

Conforme demonstrado no trecho de código, o método **pesquisar** é responsável por percorrer toda a Lista Encadeada e retornar o elemento armazenado caso a chave buscada seja localizada.

Java

```
1 public T pesquisar(T valor) {
2     No<T> atual = this.prim;           // c1: atribuição de ponteiro inicial
3
4     while (atual != null) {           // c2: avaliação condicional do laço
5         if (atual.getValor().equals(valor)) { // c3: comparação lógica de chaves
6             return atual.getValor(); // c4: retorno do elemento encontrado
7         }
8         atual = atual.getProx();        // c5: avanço do ponteiro para o próximo nó
9     }
10    return null;                       // c6: retorno de elemento não encontrado
11 }
```

É possível observar que a linha 2 inicializa a variável auxiliar *atual* recebendo a referência da cabeça da lista *this.prim*, executando uma atribuição de ponteiro de custo constante (c1). A linha 4 inicia a execução do laço de repetição *while*, cuja avaliação condicional *atual != null* verifica se a busca atingiu o final da cadeia (c2). A cada iteração do laço:

- A linha 5 invoca a comparação lógica *atual.getValor().equals(valor)* (c3). A diferenciação da busca por Nome ou por Telefone é tratada dentro do método *equals* da classe *Contato*. Se a comparação for verdadeira, a linha 6 encerra imediatamente o método retornando o objeto localizado (c4).
- Caso a comparação seja falsa, a linha 8 promove o avanço do ponteiro executando *atual = atual.getProx()* (c5).

Se a lista for percorrida integralmente sem que o valor seja localizado, o laço da linha 4 encerra e a linha 10 retorna o valor *null* (c6).

No pior dos casos, quando o contato buscado está no final da estrutura ou não pertence à coleção, o algoritmo precisa comparar a chave com todos os *n* nós armazenados.

Essa operação é algebricamente representada por $T(n) = c_1 + c_2n$. Onde *c1* representa as operações constantes de inicialização de ponteiros e retornos, e *c2n* representa o custo do laço de repetição e das comparações com *.equals()* que se repetem *n* vezes. Como buscamos analisar o comportamento assintótico de crescimento do algoritmo, ignoramos as constantes aditivas e multiplicativas, obtendo: $T(n) \approx n$.

Conforme a definição da Notação Big-O, essa relação de proporcionalidade direta leva a $O(n)$, pois o tempo que o computador leva para executar a tarefa cresce na mesma proporção que o tamanho da entrada *n*. Logo, o método *pesquisar* possui complexidade $O(n)$ no pior dos casos, pois pode ser necessário percorrer todos os elementos para confirmar a presença ou ausência do contato.

Para comprovar experimentalmente a validade do modelo teórico $T(n) = c_1 + c_2n$, analisam-se os tempos de execução coletados no Pior Caso (busca pelo último registro cadastrado via telefone) para as diferentes massas de dados n :

- Para 200.000 elementos : $C(200000) \approx 7,09ms$
- Para 400.000 elementos : $C(400000) \approx 5,82ms$
- Para 600.000 elementos : $C(600000) \approx 13ms$
- Para 800.000 elementos : $C(800000) \approx 8,59ms$

Para todas as massas de dados (200k a 800k elementos), o tempo de busca no pior caso manteve-se contido na faixa de $5,82\text{ ms}$ a $13,00\text{ ms}$, demonstrando previsibilidade operacional. A consistência da escala de tempo em milissegundos corrobora o comportamento de crescimento linear do algoritmo, ratificando a complexidade $\mathcal{O}(n)$ no pior caso.

Outro método existente no trabalho é o **remove**, que realiza a busca e a retirada de um nó da Lista Encadeada a partir de uma chave informada (como nome ou número de telefone), conforme mostra o trecho de código a seguir:

Java

```
1 public boolean remove(T valor) {
2     No<T> atual = this.prim;           // c1: inicialização do ponteiro de busca
3     No<T> ant = null;                 // c2: inicialização do ponteiro anterior
4
5     while (atual != null) {           // c3: avaliação condicional do laço
6         if (atual.getValor().equals(valor)) { // c4: comparação lógica de chaves
7             if (ant == null) {         // c5: verificação de remoção na cabeça
8                 this.prim = atual.getProx(); // c6: redefinição do ponteiro prim
9             } else {
10                 ant.setProx(atual.getProx()); // c7: desvinculação do nó intermediário
11             }
12
13             if (atual == this.ult) {    // c8: verificação de remoção na cauda
14                 this.ult = ant;        // c9: redefinição do ponteiro ult
15             }
16
17             this.quant--;               // c10: decremento do contador de nós
18             return true;               // c11: confirmação da remoção
19         }
20         ant = atual;                   // c12: avanço do ponteiro anterior
21         atual = atual.getProx();        // c13: avanço do ponteiro atual
22     }
23     return false;                     // c14: retorno de falha (elemento ausente)
24 }
```

Nas linhas 2 e 3, o método inicializa os ponteiros auxiliares *atual* apontando para a cabeça da lista *this.prim* e *ant* definido como nulo, executando atribuições de tempo constante ($c_1 + c_2$).

A linha 5 controla o percurso da estrutura através do laço *while* (c3). A cada iteração:

- A linha 6 avalia a igualdade de chaves com *.equals()* (c4).
- Ao localizar o elemento, as linhas 7 a 11 efetuam a reestruturação física dos ponteiros: se o nó removido for o primeiro da lista *ant == null*, a linha 8 atualiza *this.prim* (c6); caso contrário, a linha 10 faz o nó anterior "pular" o nó atual executando *ant.setProx(atual.getProx())* (c7).
- As linhas 13 a 15 verificam se o nó excluído era a cauda da lista (*atual == this.ult*), redefinindo a referência *this.ult = ant* se necessário (c9).
- A linha 17 decrementa a variável de controle *quant* (c10) e a linha 18 encerra a função retornando *true* (c11). Se a comparação da linha 6 for falsa, as linhas 20 e 21 promovem o avanço sequencial dos ponteiros *ant* e *atual* (c12+ c13). Se o laço for concluído sem encontrar a chave, a linha 23 retorna *false* (c14).

A desvinculação física dos ponteiros é realizada em tempo constante - $O(1)$. Mas para alcançar o nó, o algoritmo precisa realizar a busca sequencial. Portanto, no pior dos casos, quando o contato a ser removido é o último nó da lista ou não está na coleção, o laço precisa percorrer todos os anteriores. Sendo, novamente, representado algebricamente por $T(n) = c1 + c2n$.

Dessa forma, o método *remover* na Lista Não Ordenada possui ordem de complexidade $O(n)$ no pior dos casos, pois o tempo total da operação é dominado pelo custo da busca linear necessária para localizar a posição do nó antes de efetuar a troca de ponteiros.

Já na opção de **contagem de elementos**, é possível chegar ao total de elementos armazenados na lista.

O trecho do código utilizado, foi:

```
Java
1 public int quantidadeNos() {
2     return this.quant;           // c1: retorno do atributo de controle
3 }
```

Nesse método, a linha 2 acessa e retorna diretamente o valor da variável *this.quant* (c1).

Como a classe *ListaEncadeada* atualiza essa variável de controle em tempo de execução, incrementando em *adicionarNaoOrd()* e decrementando em *remover()*, o método não necessita percorrer os nós da estrutura com laços de repetição (*while* ou *for*) para contabilizar os elementos.

Dessa forma, o tempo de execução não depende do número de nós *n* armazenados na memória. Sendo representado algebricamente por $T(n) = c1$. Onde *c1* é o custo fixo e constante da instrução de retorno e leitura do atributo *quant*.

A operação de consulta à quantidade de nós executa de forma instantânea na CPU, apresentando tempos insignificantes em qualquer massa de dados:

- Para 200.000 elementos : $C(200000) < 100ns$
- Para 400.000 elementos : $C(400000) < 100ns$

- Para 600.000 elementos : $C(600000) < 100ns$
- Para 800.000 elementos : $C(800000) < 100ns$

Assim, o método *quantidadeNos* possui complexidade $O(1)$ no pior caso sob qualquer condição de preenchimento da lista, pois o valor é recuperado diretamente da memória em um único passo computacional.

b. Análise de Lista Ordenada

Uma lista ordenada é uma estrutura de dados em que a posição de cada elemento é significativa, ou seja, cada item ocupa uma posição exata (índice) dentro da estrutura, assim como em um vetor ou array. Neste trabalho, adotamos a ordenação pelo nome do contato, seguindo a ordem alfabética. No código, essa definição fica evidente com o uso do seguinte trecho:

Java

```
1 private void adicionarNaoOrd(T elem) {
2   No<T> novo = new No<T>(elem); //c1: alocação + c2: atribuição
3   if (this.prim == null) { //c3: avaliação condicional
4     this.prim = this.ult = novo; //c4: atribuição dupla (se lista vazia)
5   } else {
6     this.ult.setProx(novo); //c5: atribuição de referência (se lista com nós)
7     this.ult = novo; //c6: redefinição do ponteiro cauda
8   }
9   this.quant++; //c7: incremento (leitura + soma + escrita)
10 }
```

O Comparator atua como a regra que determina qual elemento deve vir primeiro. A inserção é feita pelo método adicionar(), que internamente chama adicionarOrd(). Este percorre a lista comparando os nomes, encontra a posição adequada e insere o novo nó, mantendo a ordenação.

Java

```
private void adicionarOrd(T elem) {
  No<T> novo = new No<T>(elem); //c1: alocação + c2: atribuição
  No<T> atual = this.prim; //c3: atribuição
  No<T> ant = null; //c4: atribuição
  if (this.prim == null) { //c5: avaliação condicional
    this.prim = this.ult = novo; //c6: atribuição dupla (se lista vazia)
  } else {
    while (atual != null && comparador.compare(atual.getValor(), elem) < 0) { //c7: comparação (n
      iterações)
    }
  }
}
```



```

    ant = atual;          //c8: atribuição
    atual = atual.getProx(); //c9: avanço
}
if (ant == null) {       //c10: verificação condicional
    novo.setProx(this.prim); //c11: atribuição
    this.prim = novo;      //c12: atribuição
} else if (atual == null) { //c13: verificação condicional
    this.ult.setProx(novo); //c14: atribuição
    this.ult = novo;       //c15: atribuição
} else {
    ant.setProx(novo);     //c16: atribuição
    novo.setProx(atual);   //c17: atribuição
}
}
this.quant++;           //c18: incremento
}

```

Se existem n elementos, no pior dos casos o novo contato pode precisar ser comparado com todos eles. Essa operação é representada por $T(n) = C1 + C2n$, em que $C1$ representa operações constantes e $C2n$, as que se repetem para cada elemento. Como buscamos comparar o crescimento, ignoramos as constantes, obtendo $T(n)$ aproximadamente igual a n . Logo, a inserção ordenada possui complexidade $O(n)$ no pior caso, pois pode ser necessário percorrer todos os elementos para encontrar a posição correta.

Vale destacar que a inserção de dados em uma lista encadeada possui particularidades importantes. Na lista não ordenada, a inserção ocorre sempre no final da estrutura, o que permite aproveitar a referência `ult` para realizar a operação em tempo constante, ou seja, $O(1)$. Nesse caso, não há necessidade de percorrer a lista, pois o novo nó é simplesmente encadeado após o último elemento. Já na lista ordenada, essa mesma operação exige que o algoritmo percorra a estrutura para localizar a posição correta do novo elemento, o que resulta em $O(n)$ no pior caso. Essa diferença é relevante porque mostra que a escolha entre lista ordenada e não ordenada impacta diretamente o custo de inserção, mesmo quando ambas utilizam a mesma estrutura de nós encadeados. No entanto, essa vantagem da lista não ordenada na inserção não se estende às buscas, que permanecem lineares em ambos os casos, como será discutido a seguir.

A busca utiliza o método `pesquisar()`:

```

Java
No<T> atual = this.prim;
while (atual != null) {
    if (atual.getValor().equals(valor)) {
        return atual.getValor();
    }
}

```

```
}  
    atual = atual.getProx();  
}
```

É importante esclarecer um ponto central da nossa análise: o fato de uma lista ser ordenada não implica que sua busca tenha complexidade $O(\log n)$. Para que isso fosse verdade, seria necessário utilizar busca binária, acessando o elemento central, comparando-o e descartando metade da lista a cada iteração. No entanto, o código implementado percorre a lista elemento por elemento, de forma sequencial. Como consequência, o número de elementos examinados cresce proporcionalmente a n , ou seja, $O(n)$ no pior caso. Assim, ordenar uma lista não altera a complexidade de sua busca quando o método utilizado é sequencial.

A remoção segue o mesmo padrão. O método `remover()` percorre a lista desde o primeiro elemento até encontrar o valor procurado, mantendo uma referência ao nó anterior para ajustar o encadeamento:

```
Java  
No<T> atual = this.prim;  
No<T> ant = null;  
while (atual != null) {           //c1: verificação condicional (n iterações)  
    if (atual.getValor().equals(valor)) { //c2: comparação equals  
        if (ant == null) {           //c3: verificação condicional  
            this.prim = atual.getProx(); //c4: atribuição  
        } else {  
            ant.setProx(atual.getProx()); //c5: atribuição  
        }  
        if (atual == this.ult) { //c6: verificação condicional  
            this.ult = ant;      //c7: atribuição  
        }  
        this.quant--;           //c8: decremento  
        return true;           //c9: retorno  
    }  
    ant = atual;                 //c10: atribuição  
    atual = atual.getProx();      //c11: avanço  
}  
return false;                   //c12: retorno
```

No pior caso, quando o elemento a ser removido é o último, o algoritmo percorre todos os n elementos, resultando novamente em $O(n)$. Embora exista uma referência `ult` para o último nó, ela não permite localizar o elemento e seu antecessor; o percurso cresce proporcionalmente a n .

Determinado que inserção, busca por nome, busca por telefone e remoção por telefone possuem, em seus piores casos, complexidade $O(n)$, passamos à comparação com os testes empíricos. Para isso, consideramos n como variável de entrada e realizamos testes com 200.000, 400.000, 600.000 e 800.000 registros, avaliando quatro operações: leitura, busca por nome, busca por telefone e remoção por telefone, conforme a Tabela 1.

Para uma lista encadeada, todas essas operações possuem comportamento linear no pior caso, pois o algoritmo pode precisar percorrer os elementos sequencialmente. Assim, espera-se uma relação aproximada $T(n) = a + bn$, em que a é o custo fixo e b , o custo por elemento.

Como os testes buscaram o último elemento da lista, o algoritmo percorre praticamente todos os n elementos antes de encontrá-lo. O número de comparações cresce, portanto, de forma linear:

- 200.000 elementos, aproximadamente 200.000 comparações
- 400.000 elementos, aproximadamente 400.000 comparações
- 600.000 elementos, aproximadamente 600.000 comparações
- 800.000 elementos, aproximadamente 800.000 comparações

A busca por telefone utiliza a mesma função `pesquisar()`, alterando apenas o critério de comparação no método `equals()`. O percurso permanece idêntico, mantendo $T(n)$ em $O(n)$. O mesmo vale para a remoção por telefone.

```
Java
if (this == obj)                //c1: comparação de referência
    return true;                //c2: retorno
if (obj == null || getClass() != obj.getClass()) //c3: verificação de nulo + classe
    return false;               //c4: retorno
Contato outro = (Contato) obj;   //c5: cast
if (outro.telefone.isEmpty()) {  //c6: verificação condicional
    return this.nome.equalsIgnoreCase(outro.nome); //c7: comparação por nome
}
return this.telefone.equals(outro.telefone); //c8: comparação por telefone
```

Os valores obtidos nos experimentos permitem relacionar o comportamento previsto com o observado. Na leitura dos dados, houve crescimento significativo conforme o volume aumenta: 113 ms (200 mil), 645 ms (400 mil), 1.596 ms (600 mil) e 3.311 ms (800 mil). Esse crescimento é compatível com o aumento do trabalho realizado pelo programa.

Nas operações de busca por nome, os tempos foram 447 ns, 455 ns, 450 ns e 232 ns; na busca por telefone, 120 ns, 385 ns, 659 ns e 177 ns; e na remoção, 259 ns, 429 ns, 755 ns e 182 ns. Embora esses valores não apresentem crescimento estritamente proporcional a n , devem ser analisados em conjunto com a complexidade teórica: em todas as operações, o código

percorre a lista sequencialmente e, nos testes, o elemento procurado ou removido está no final, fazendo com que o número de elementos percorridos seja proporcional a n .

Teoricamente, para n_1 igual a 200.000 e n_2 igual a 800.000, temos n_2 dividido por n_1 igual a 4. Como as operações são lineares, o custo esperado também deveria crescer aproximadamente 4 vezes, caracterizando $T(n) = a + bn$ e, portanto, $T(n)$ em $O(n)$.

Entretanto, os tempos experimentais não precisam seguir exatamente essa proporção, pois o tempo medido não depende apenas da quantidade de elementos percorridos. Fatores como execução da JVM, otimizações do compilador JIT, gerenciamento de memória, coleta de lixo, cache do processador e outras atividades do sistema interferem na medição. Dessa forma, os resultados experimentais devem ser interpretados como uma observação do comportamento prático, enquanto a análise matemática estabelece a tendência de crescimento.

Tabela 1 - TABELA COMPARATIVA DOS TESTES EMPÍRICOS

TABELA COMPARATIVA DOS TESTES EMPÍRICOS				
Execução em Tempo Quant. Dados	Leitura	Busca Nome → Número de Telefone	Busca Número de Telefone → Nome	Remoção de dados
200 k	113 ms	447 nanosegundos	120 nanosegundos	259 nanosegundos
400 k	645 ms	455 nanosegundos	385 nanosegundos	429 nanosegundos
600 k	1.596 ms	450 nanosegundos	659 nanosegundos	755 nanosegundos
800 k	3.311 ms	232 nanosegundos	177 nanosegundos	182 nanosegundos

Tabela 2 -TABELA DAS OPERAÇÕES

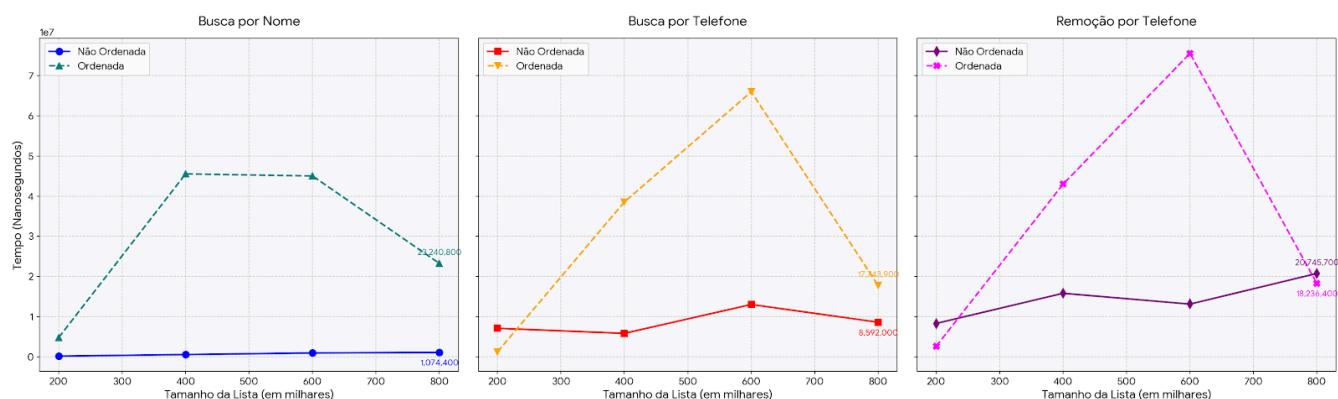
Operação	Lista Não Ordenada	Lista Ordenada
Inserção (adicionar)	$O(1)$	$O(n)$
Busca (pesquisar)	$O(n)$	$O(n)$
Remoção (remover)	$O(n)$	$O(n)$
Contagem (quantidadeNos)	$O(1)$	$O(1)$

4. Análise Empírica de complexidade de Algoritmos

Conforme apresentado na tabela comparativa e nos gráficos a seguir, os tempos de execução coletados empiricamente confirmam e complementam a análise matemática prévia, avaliando o comportamento da estrutura em função do tamanho da entrada.

Imagem 1 - COMPARAÇÃO ORDENADA X NÃO ORDENADA

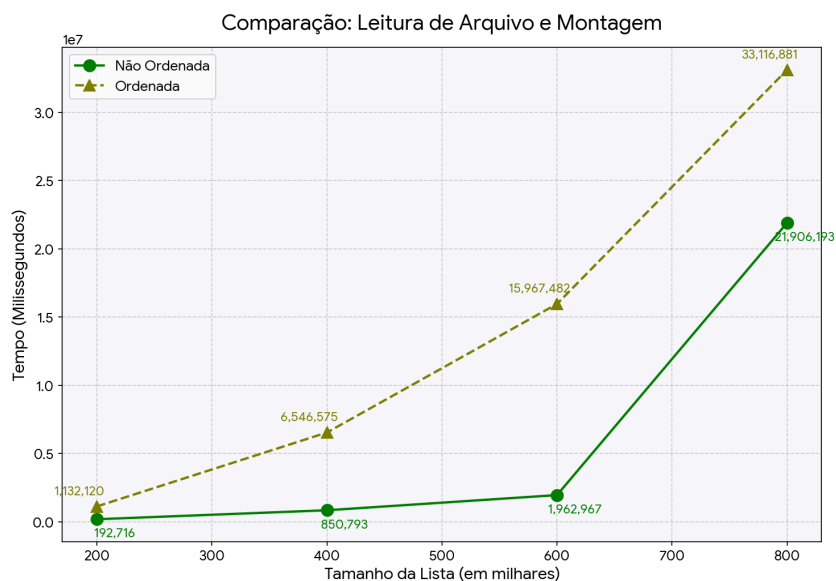
Comparação: Lista Não Ordenada vs. Ordenada (Mesma Escala - Nanosegundos)



Ao analisar os gráficos referentes às operações em memória, o contraste entre as implementações evidencia o impacto prático das análises matemáticas descritas anteriormente. Na lista não ordenada, a busca pelo último elemento inserido apresentou o crescimento linear constante esperado, ilustrando perfeitamente o pior cenário de uma varredura sequencial e validando a complexidade $O(n)$.

Contudo, na lista ordenada, as buscas e remoções apresentaram tempos altamente oscilantes e atípicos, a exemplo da queda abrupta de tempo observada no cenário de 800 mil registros. Esse fenômeno ocorre puramente por contingência posicional: a ordenação alfabética deslocou contatos lidos no fim do arquivo para posições iniciais da memória, mascarando o pior caso. Esse comportamento comprova empiricamente a constatação teórica prévia de que a ordenação da lista, por si só, não reduz a complexidade da busca quando o método implementado utiliza varredura sequencial em vez de busca binária. Fica evidente que realizar operações por chaves secundárias (como o telefone) em uma estrutura ordenada por chave primária (nome) resulta no mesmo custo exaustivo $O(n)$ de uma estrutura não ordenada.

Imagem 2 - COMPARAÇÃO LEITURA



Por fim, o gráfico referente ao tempo total de leitura e montagem evidencia o gargalo mais crítico do sistema e a principal divergência de desempenho entre as abordagens. A lista não ordenada apresentou um tempo de execução significativamente menor (3.311 ms para 800 mil registros) e mais estável, comportamento reflexo da complexidade constante $O(1)$ de sua inserção no final da cauda. Em contrapartida, a montagem da lista ordenada assumiu uma curva de crescimento exponencial.

Isso corrobora de forma exata o previsto pela análise matemática: como a inserção ordenada exige varreduras contínuas para localizar a posição correta, operando individualmente em $O(n)$, o custo computacional total para encadear repetidamente n elementos explode para a ordem de grandeza de $O(n^2)$. Essa ineficiência multiplicou os custos operacionais, justificando o salto dramático no tempo de processamento, que saltou de pouco mais de 3 segundos na lista não ordenada para mais de 33 milhões de milissegundos (aproximadamente 9 horas) na lista ordenada. Tais resultados provam, em definitivo, que a manutenção dinâmica de ordenação sequencial torna-se insustentável para grandes volumes de dados.